المدرسة العليا للتكنولوجيا - الداخلة
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵓⵙⵙⵏⴰⵡⵉⵜ - ⴷⴰⵅⵍⴰ
ÉCOLE SUPÉRIEURE DE TECHNOLOGIE - DAKHLA

Programmation objet avancée

2ème année GI

Pr. Naoual Nassiri

N.Nassiri@uiz.ac.ma

Année universitaire 2023-2024

Introduction

Le mot polymorphisme vient du grecque : **poly** (pour plusieurs) et **morph** (forme). Il veut dire qu'une même chose peut avoir différentes formes. Nous avons déjà vue cette notion avec la redéfinition des méthodes. Une même méthode peut avoir différentes définitions suivant la classe où elle se trouve.

Exemple introductif

Soient les classes **Personne**, **Etudiant** et **EtudiantEtranger** définies dans les chapitres précédents :

```
class Personne {
  ...
}
class Etudiant extends Personne {
     ...
}
class EtudiantEtranger extends Etudiant {
    private String nationalite;
     ...
}
```

Exemple introductif

Puisque un étudiant étranger est lui aussi un étudiant, au lieu de définir 2 tableaux :

```
Etudiant [] etudiants = new Etudiant[30];
EtudiantEtranger[] etudiantsEtrangers = new EtudiantEtranger[10];
```

on pourra définir un seul tableau comme suit :

```
Etudiant [] etudiants = new Etudiant[40];
```

Exemple introductif

Avant d'utiliser le tableau précédent, considérons la déclaration :

Personne personne;

Nous avons vu que les instructions suivantes sont toutes valides :

personne = **new** Personne();
personne = **new** Etudiant();
personne = **new** EtudiantEtranger();

De la même façon on peut initialiser le tableau de la façon suivante :

```
for(int int i=0; i<10; i++)
    etudiants[i] = new Etudiant();

etudiants[10] = new EtudiantEtranger();
etudiants[11] = new EtudiantEtranger();
...
```

Liaison dynamique

Considérons la classe **B** qui hérite de la classe **A** :

```
class A{
  public void message(){
     System.out.println("Je suis dans la classe A");
  }
}

class B extends A{
  public void message(){
    System.out.println("Je suis dans la classe B");
  }
  public void f(){
    System.out.println("Methode f de la classe B");
  }
}
```

Liaison dynamique

Considérons les instructions :

```
public static void main(String[] args) {
    A a = new A();
    B b = new B();
    a.message();
    b.message();
    b.f();
    a = new B();
    a.message();
}
```

Liaison dynamique

Dans l'exécution on aura le résultat suivant :

Je suis dans la classe A
Je suis dans la classe B
Methode f() de la classe B
Je suis dans la classe B

- Lorsqu'une méthode est redéfinie (s'est spécialisée), c'est la version la plus spécialisée qui est appelée. La recherche de la méthode se fait dans la classe réelle de l'objet. La recherche s'est fait lors de l'exécution et non lors de la compilation. Ce processus s'appelle la **liaison dynamique**.
- Il s'appelle aussi : **liaison tardive, dynamic binding, late-binding** ou **run-time binding**.

Remarques

- Dans les instructions précédentes, la dernière instruction «**a.message();**» fait appel à la méthode « message() » de la classe **B**.
- Si on ajoute l'instruction : **a. f () ;** après l'instruction : **a = new B();** on aura une erreur de compilation, du fait que la méthode « f() » n'est pas implémentée dans la classe de déclaration de l'objet « a » même si la classe réelle (la classe B) possède « f() ».
- Pour éviter l'erreur précédente, il faut faire un cast : **((B) a). f () ;**

Remarques

- La visibilité d'une méthode spécialisée peut être augmentée (par exemple de protected vers public) mais elle ne peut pas être réduite (par exemple de public vers private)

Exemple

```
class Etudiant {
    public void message(){
        System.out.println("Je suis un etudiant");
    }
}

class EtudiantEtranger extends Etudiant {
    public void message(){
        System.out.println("Je suis un etudiant
            etranger");
    }
}
```

Exemple

```
public class MethodesInstances {
    public static void main(String[] args){
        Etudiant e = new Etudiant();
        e.message();

        e = new EtudiantEtranger();
        e.message();
    }
}
```

L'exécution du programme précédent donnera :

Je suis un etudiant

Je suis un etudiant etranger

Méthodes de classes

Si on modifie la méthode « **message()** » de la classe **Etudiant**, en la rendant statique :

```
class Etudiant {
    public static void message(){
        System.out.println("Je suis un etudiant");
    }
}

class EtudiantEtranger extends Etudiant {
    public static void message(){
        System.out.println("Je suis un etudiant
            etranger");
    }
}
```

Méthodes de classes

```
public class MethodesInstances {
    public static void main(String[] args){
        Etudiant e = new Etudiant();
        e.message();
        e = new EtudiantEtranger();
        e.message();
    }
}
```

- Alors l'exécution du programme précédent donnera :

  Je suis un etudiant

  Je suis un etudiant

- C'est la classe du type de déclaration qui est utilisée (ici Etudiant) et non du type réel de l'objet (ici EtudiantEtranger).

Abstraction

- Reprenons les classes **Personne**, **Etudiant** et **EtudiantEtranger** et ajoutons aux classes **Etudiant** et **EtudiantEtranger** la méthode «**saluer()** » :

Abstraction

```
class Etudiant extends Personne {
    ...
    public void saluer() {
        System.out.println("Assalam alaikoum");
    }
}
class EtudiantEtranger extends Etudiant {
    private String nationalite;
    ...
    public void saluer() {
        System.out.println("Bonjour");
    }
}
```

- Puisque la méthode « saluer() » est définie dans les deux classes **Etudiant** et **EtudiantEtranger**, on souhaite la définir dans la classe **Personne**.

Solution 1

Une première solution consiste à la définir comme suit :

```
class Personne {
    ...
    public void saluer() {
    }
}
```

Cette solution est mauvaise du fait que toute classe qui héritera de **Personne** pourra appeler cette méthode (qui ne fait rien !).

Solution 2

- Une deuxième solution consiste à rendre la méthode « saluer() » abstraite dans **Personne** et par conséquent, obliger chaque classe qui hérite de **Personne** à définir sa propre méthode.

Remarques

- Une méthode abstraite :
  - ne doit contenir que l'entête et doit être implémenté dans les sous classes ;
  - doit être public (ne peut pas être privée) ;
  - est déclarée comme suit :

```
public abstract typeRetour methAbstr(args);
```

- Une méthode statique ne peut pas être abstraite.
- Une classe qui contient une méthode abstraite doit être elle aussi abstraite et doit être déclarée comme suit :

```
public abstract class nomClasse{ ... }
```

Remarques

- Une classe abstraite ne peut pas être utilisée pour instancier des objets.

  Une instruction telle que :

```
obj=new nomClasse();
```

est incorrecte (avec nomClasse est une classe abstraite).

- Une sous-classe d'une classe abstraite doit implémenter toutes les méthodes abstraites sinon elle doit être déclarée abstraite.

Solution 2 :

La classe **Personne** devient comme suit :

```
abstract class Personne {
    private String nom, prenom;
    public abstract void saluer();
    //...
}
```

Exemple 2

- Considérons les classes **Cercle** et **Rectangle** qui sont des sous classes de la classe **FigureGeometrique**. Les surfaces d'un cercle et d'un rectangle ne sont pas calculées de la même façon.

➔ Une solution consiste à définir dans la classe **FigureGeometrique** une méthode abstraite « surface() » et obliger les classes **Cercle** et **Rectangle** à implémenter cette méthode.

Exemple 2

```
abstract class FigureGeometrique {
    public abstract double surface();
}

class Cercle extends FigureGeometrique {
    private double rayon;

    public Cercle(double rayon) {
        this.rayon = rayon;
    }

    public double surface() {
        return Math.PI * rayon * rayon;
    }
}
```

Exemple 2

```
class Rectangle extends FigureGeometrique {
    public double largeur, longueur;

    public Rectangle(double large, double longue) {
        this.largeur = large;
        this.longueur = longue;
    }

    public double surface() {
        return largeur * longueur;
    }
}
```